

AdMaya.ai

Technical Implementation Plan, Architecture, Roadmap & Starter Code

React frontend → production-ready AI video generation platform

Prepared from the AI Provider Integration Playbook · September 2026

Recommended MVP decision Keep the existing React frontend. Add a TypeScript backend using NestJS + Fastify, PostgreSQL + Prisma, Redis + BullMQ, S3-compatible storage, Razorpay, Firebase Authentication, and a provider-adapter layer. Launch first with fal.ai/Seedance, then add Runway and HeyGen.

1. Executive recommendation

The uploaded playbook is directionally correct: the browser must never call video providers directly; render jobs are asynchronous; provider URLs are temporary; and a clean adapter layer is the core long-term asset. The revised product concept is a template marketplace where visual settings are locked, the user edits only the script, pays per render, and receives a watermarked preview/export.

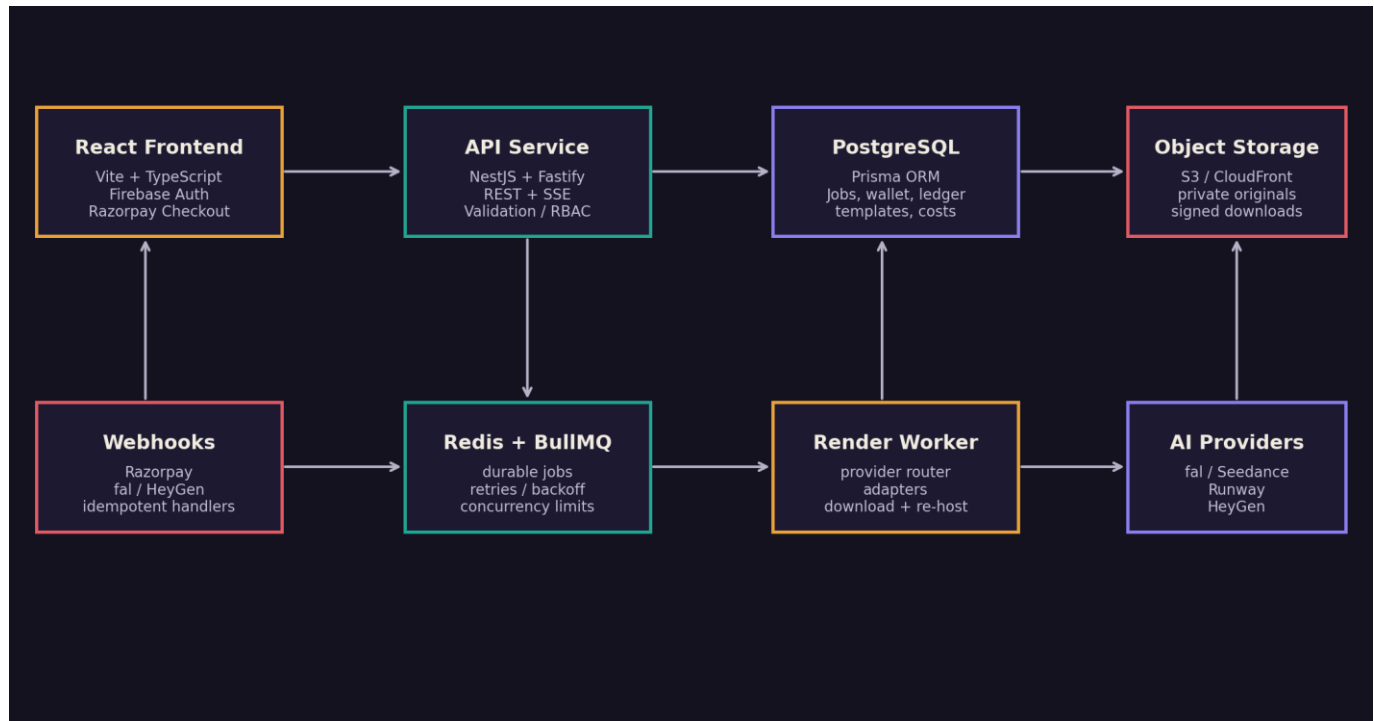
Decision	Recommendation	Reason
Frontend	Existing React app; migrate to TypeScript incrementally	Preserves completed UI and adds safer API contracts.
Backend	Node.js + TypeScript + NestJS using Fastify	Structured modules, validation, testing, webhooks, and worker-friendly code.
Database	PostgreSQL + Prisma ORM	Best fit for jobs, templates, wallets, immutable ledger entries, orders, and provider-cost reporting.
Queue	Redis + BullMQ	Video generation is long-running; workers need retries, backoff, concurrency and recovery.
Storage	Amazon S3 + CloudFront, or Cloudflare R2 for MVP	Re-host provider outputs and issue short-lived signed download links.
Payments	Razorpay Orders + Checkout + verified webhooks	Strong India fit; payment success must be verified server-side.
Authentication	Firebase Authentication	Fast React integration; backend verifies ID tokens. Replaceable later.
MVP provider	fal.ai queue API routing to Seedance	One integration, async queue, webhook support, fast template experimentation.
Next providers	Runway direct, then HeyGen	Runway for promo/feature motion; HeyGen for avatar-led templates.
Deployment	AWS ECS Fargate + RDS + ElastiCache + S3; use Railway/Render only for private beta	Workers need durable processes and predictable resource controls.

Do not build MCP into the customer product Use REST/SDK provider APIs. MCP can be an optional internal template-testing tool, but it should not sit in the production user render path.

2. Product scope inferred from the frontend and playbook

- Template gallery: eight templates with provider, engine, duration, aspect ratio, styling, camera, pacing, prompt and selling price.
- Locked-template studio: the user can edit only the script and permitted brand inputs.
- Pay-per-render checkout: reserve or charge credits before provider submission.
- Asynchronous lifecycle: payment_pending → queued → submitted → processing → completed/failed.
- Watermarked preview and controlled export; permanent clean master handling must be defined as a business policy.
- Provider routing: changing a template provider must be configuration, not a frontend or orchestration rewrite.
- Financial observability: record charge, refund, actual provider cost and gross margin per render.

3. Target architecture



Request flow: React authenticates the user and requests an order. Razorpay completes payment and the backend verifies the signed webhook. The API creates an immutable wallet ledger entry and enqueues a render. A BullMQ worker loads the locked template, builds the provider payload, submits the asynchronous provider job, persists every state change, accepts webhook callbacks or polls as fallback, downloads the result, writes it to object storage, and marks the job complete. The frontend receives status through polling for MVP and Server-Sent Events later.

4. Recommended technology stack

Layer	Primary choice	MVP alternative	Notes
Web UI	React + Vite + TypeScript	Keep current JavaScript briefly	Add TanStack Query, React Router, Zod, Firebase SDK and Razorpay Checkout.
API	NestJS + Fastify	Fastify alone	NestJS is useful because provider adapters, webhooks, orders and jobs become separate modules.
Validation	Zod or class-validator	Manual validation not recommended	Share DTO types through an internal package.
Database	PostgreSQL	Supabase Postgres / Neon / Prisma Postgres	Use transactions and row locks for wallet correctness.
ORM	Prisma	Drizzle	Prisma gives typed models and migrations for a TypeScript codebase.
Queue	BullMQ + Redis	Cloud queue later	Separate API and worker processes. Configure retries and idempotency.
Object storage	S3 + CloudFront	Cloudflare R2	Keep buckets private; download through short-lived signed URLs.
Payments	Razorpay	Stripe for international expansion	Treat webhook as source of truth; never trust client success alone.
Auth	Firebase Auth	Clerk / Auth0 / Cognito	Verify tokens in backend; store external UID in User.

Layer	Primary choice	MVP alternative	Notes
Observability	OpenTelemetry + Sentry + structured logs	Sentry only for MVP	Add job/provider/trace IDs to every log.
CI/CD	GitHub Actions	GitLab CI	Lint, typecheck, tests, migration check, image build, deploy.
Infrastructure	AWS ECS/RDS/ElastiCache/S3/CloudFront	Railway/Render + managed DB/Redis	Move to AWS before public scale or high rendering spend.

5. Provider strategy and rollout

Template type	MVP engine	Later option	Routing policy
Unbox / UGC product	Seedance via fal.ai	BytePlus direct	Route high-volume templates direct only after volume justifies integration cost.
Before / after	Seedance first	Higgsfield	Promote to premium only after A/B quality and margin testing.
Application walkthrough	Deterministic compositor + Runway overlays	Seedance	Do not generate UI screens with AI; record actual product UI.
Founder / customer avatar	HeyGen	D-ID comparison	Require explicit avatar/voice consent and retain audit evidence.
Feature spotlight / offers	Runway	Seedance	Optimize for low cost and short duration.
Street reaction	Seedance or Kling via aggregator	Kling direct	Keep documentary look as a template-level prompt and reference set.
Flagship premium	Not in MVP	Vevo on Vertex AI	Use only when premium selling price covers the materially higher cost.

The playbook maps Seedance to high-volume UGC, HeyGen to talking presenters, Runway to low-cost stylized clips, Higgsfield to cinematic work, and Vevo to premium outputs. Current provider documentation continues to use asynchronous task or callback patterns. fal recommends its persistent queue with status polling or webhooks; Runway returns asynchronous task IDs; HeyGen supports callback URLs on video creation; and Vertex AI video generation returns long-running operations. [1–4]

6. Core data model

Use integers for money in the smallest unit, such as paise. Keep the wallet ledger append-only. A balance is a derived or transactionally updated value, never a client-supplied number.

```
enum RenderStatus {
  PAYMENT_PENDING
  QUEUED
  SUBMITTED
  PROCESSING
  COMPLETED
  FAILED
  REFUNDED
}

model Template {
  id          String   @id @default(cuid())
  slug        String   @unique
  title       String
  provider    String
  providerModel String
  durationSec  Int
  aspectRatio String
  lockedPrompt Json
  inputSchema  Json
  sellPricePaise Int
  active       Boolean @default(true)
  version      Int     @default(1)
  renders      RenderJob[]
}

model RenderJob {
  id          String   @id @default(cuid())
  userId      String
  templateId   String
  templateVersion Int
  status       RenderStatus @default(PAYMENT_PENDING)
  script       String
  provider     String
  providerJobId String?
  providerCostMicros BigInt?
  outputObjectKey String?
  errorCode    String?
  errorMessage  String?
  idempotencyKey String @unique
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
  template     Template @relation(fields:[templateId], references:[id])
}

model WalletLedger {
  id          String   @id @default(cuid())
  userId      String
  type        String   // CREDIT_PURCHASE, RENDER_DEBIT, REFUND
  amountPaise Int      // positive credit, negative debit
  referenceType String
  referenceId  String
  idempotencyKey String @unique
  createdAt    DateTime @default(now())
}

model PaymentOrder {
  id          String   @id @default(cuid())
  userId      String
  razorpayOrderId String @unique
  razorpayPaymentId String? @unique
  amountPaise Int
  status      String
  createdAt    DateTime @default(now())
}
```

7. API contract

Method	Route	Purpose
GET	/v1/templates	List active public templates; never return secret provider prompts.
GET	/v1/templates/:slug	Return public locked settings and allowed user inputs.
POST	/v1/payments/orders	Create Razorpay order or credit-pack purchase.
POST	/v1/webhooks/razorpay	Verify raw-body signature and credit wallet exactly once.
POST	/v1/renders	Validate wallet, reserve/debit credit, snapshot template version, queue render.
GET	/v1/renders/:id	Return status, progress, safe error message and preview URL.
POST	/v1/webhooks/fal	Provider completion callback; verify authenticity and enforce idempotency.
POST	/v1/webhooks/heygen	HeyGen completion callback.
GET	/v1/assets/:id/download	Authorize ownership and return short-lived signed URL.
POST	/v1/admin/templates	Create versioned template; admin role only.

8. Monorepo structure

```

admaya/
├─ apps/
│  └─ web/           # your existing React frontend
│  └─ api/           # NestJS HTTP API
│  └─ worker/        # BullMQ render worker
├─ packages/
│  └─ contracts/     # shared Zod schemas and TypeScript types
│  └─ database/      # Prisma schema and client
│  └─ providers/     # adapter interface + fal/runway/heygen
│  └─ config/        # typed environment validation
├─ infra/
│  └─ docker-compose.yml # Postgres + Redis for local development
│  └─ terraform/      # production AWS infrastructure
├─ .github/workflows/ci.yml
└─ pnpm-workspace.yaml

```

9. Backend starter implementation

9.1 Provider interface

```

export type ProviderState =
  | "QUEUED" | "PROCESSING" | "COMPLETED" | "FAILED";

export interface RenderPayload {
  jobId: string;
  script: string;
  template: { providerModel: string; lockedPrompt: unknown };
  callbackUrl: string;
}

export interface ProviderAdapter {
  submit(payload: RenderPayload): Promise<{ providerJobId: string }>;
  status(providerJobId: string): Promise<{ state: ProviderState; progress?: number }>;
  result(providerJobId: string): Promise<{ temporaryUrl: string; durationSec?: number; costMicros?: bigint }>;
  cancel?(providerJobId: string): Promise<void>;
}

```

9.2 fal.ai adapter

```

import { fal } from "@fal-ai/client";
fal.config({ credentials: process.env.FAL_KEY! });

export class FalSeedanceAdapter implements ProviderAdapter {
  async submit(payload: RenderPayload) {
    const model = payload.template.providerModel;
    const response = await fal.queue.submit(model, {
      input: buildFalInput(payload),
      webhookUrl: payload.callbackUrl,
    });
    return { providerJobId: response.request_id };
  }

  async status(providerJobId: string) {
    const s = await fal.queue.status(process.env.FAL_MODEL!, {
      requestId: providerJobId, logs: true,
    });
    const map = { IN_QUEUE: "QUEUED", IN_PROGRESS: "PROCESSING", COMPLETED: "COMPLETED" } as const;
    return { state: map[s.status] ?? "FAILED" };
  }
}

```

```

async result(providerJobId: string) {
  const r: any = await fal.queue.result(process.env.FAL_MODEL!, { requestId: providerJobId });
  const url = r.data?.video?.url ?? r.data?.video_url;
  if (!url) throw new Error("Provider returned no video URL");
  return { temporaryUrl: url };
}

```

Important Model endpoint IDs and exact input/output fields vary by provider model. Store them in template/provider configuration and validate them against the provider's current API reference before deployment.

9.3 Render creation with transactional debit

```

async createRender(userId: string, dto: CreateRenderDto) {
  return this.prisma.$transaction(async tx => {
    const template = await tx.template.findFirstOrThrow({
      where: { id: dto.templateId, active: true },
    });

    const balance = await getWalletBalanceForUpdate(tx, userId);
    if (balance < template.sellPricePaise) throw new PaymentRequiredException();

    const job = await tx.renderJob.create({ data: {
      userId, templateId: template.id, templateVersion: template.version,
      script: dto.script, provider: template.provider,
      idempotencyKey: dto.idempotencyKey, status: "QUEUED",
    }});

    await tx.walletLedger.create({ data: {
      userId, type: "RENDER_DEBIT", amountPaise: -template.sellPricePaise,
      referenceType: "RENDER_JOB", referenceId: job.id,
      idempotencyKey: `render-debit:${job.id}`,
    }});

    await this.renderQueue.add("submit", { renderJobId: job.id }, {
      jobId: job.id, attempts: 5,
      backoff: { type: "exponential", delay: 5000 },
      removeOnComplete: 500, removeOnFail: 2000,
    });
    return job;
  });
}

```

9.4 Worker orchestration

```

new Worker("render", async queueJob => {
  const job = await prisma.renderJob.findUniqueOrThrow({
    where: { id: queueJob.data.renderJobId }, include: { template: true },
  });
  if (["COMPLETED", "REFUNDED"].includes(job.status)) return;

  const adapter = providerRegistry.get(job.provider);
  if (!job.providerJobId) {
    const submitted = await adapter.submit({
      jobId: job.id, script: job.script, template: job.template,
      callbackUrl: `${env.PUBLIC_API_URL}/v1/webhooks/${job.provider}`,
    });
    await prisma.renderJob.update({ where: { id: job.id }, data: {
      providerJobId: submitted.providerJobId, status: "SUBMITTED",
    }});
  }
}, { connection: redis, concurrency: 4 });

```

9.5 Completion handler and re-hosting

```
async function completeRender(jobId: string, temporaryUrl: string) {
  const response = await fetch(temporaryUrl);
  if (!response.ok || !response.body) throw new Error("Video download failed");
  const objectKey = `renders/${jobId}/master.mp4`;
  await s3.send(new PutObjectCommand({
    Bucket: env.S3_BUCKET, Key: objectKey, Body: Readable.fromWeb(response.body as any),
    ContentType: "video/mp4", ServerSideEncryption: "AES256",
  }));
  await prisma.renderJob.update({
    where: { id: jobId }, data: { status: "COMPLETED", outputObjectKey: objectKey },
  });
}
```

10. Payments and wallet correctness

For the current Indian pay-per-video concept, use Razorpay Orders and Checkout. The browser can display Checkout, but only the backend may credit the wallet or unlock rendering. Razorpay recommends webhooks for server-side payment state and requires webhook-secret validation. [5]

```
// Raw-body webhook route: do not JSON-transform before verification.
@Post("webhooks/razorpay")
async razorpayWebhook(@Req() req: RawBodyRequest<Request>, @Headers("x-razorpay-signature") sig: string) {
  const expected = createHmac("sha256", env.RAZORPAY_WEBHOOK_SECRET)
    .update(req.rawBody!).digest("hex");
  if (!timingSafeEqual(Buffer.from(expected), Buffer.from(sig)))
    throw new UnauthorizedException();

  const event = JSON.parse(req.rawBody!.toString("utf8"));
  if (event.event === "order.paid") {
    await paymentService.creditOrderExactlyOnce(event.payload.order.entity);
  }
  return { ok: true };
}
```

Required money rules

- All webhook handlers are idempotent; repeated delivery must not add credits twice.
- Debit and render creation occur in one database transaction.
- Failure refund is a new positive ledger row, never deletion of the original debit.
- Provider cost is stored separately from customer price.
- No render begins until payment or sufficient wallet credit is verified.
- Use a deterministic idempotency key for order creation and render submission.

11. Integrating the existing React frontend

Keep the current component design, but replace simulated timers with API calls and server state. Use TanStack Query for templates and render status. The first MVP can poll every four seconds; SSE can be introduced after the core path is stable.

```
// web/src/api/renderers.ts
export async function createRender(input: { templateId:string; script:string; idempotencyKey:string }) {
  const token = await auth.currentUser?.getIdToken();
  const res = await fetch(`${import.meta.env.VITE_API_URL}/v1/renderers`, {
    method: "POST",
    headers: { "Content-Type":"application/json", Authorization:`Bearer ${token}` },
    body: JSON.stringify(input),
  });
  if (!res.ok) throw new Error((await res.json()).message ?? "Render request failed");
  return res.json();
}

export function useRender(renderId?: string) {
  return useQuery({
    queryKey:["render", renderId],
    queryFn:()=>api.getRender(renderId!), enabled:!!renderId,
    refetchInterval:q => ["COMPLETED","FAILED","REFUNDED"].includes(q.state.data?.status) ? false : 4000,
  });
}
```

```
// Replace the prototype Generate button flow
const mutation = useMutation({ mutationFn:createRender });
const generate = async () => {
  const job = await mutation.mutateAsync({
    templateId: activeTemplate.id, script, idempotencyKey: crypto.randomUUID(),
  });
}
```

```
setRenderId(job.id);
};
```

12. Security, moderation and abuse controls

Control	Implementation
Secrets	AWS Secrets Manager or deployment secret store. Never expose provider keys through Vite variables.
Authentication	Verify Firebase ID token in a NestJS guard; use role claims for admin endpoints.
Authorization	Every render and asset query includes userId ownership checks.
Input validation	Script length, language, prohibited content, URL allowlists and template-specific schemas.
Prompt integrity	Server builds the final prompt; client submits only permitted variables.
Webhook security	Signature verification, timestamp/replay checks where supported, raw-body handling and idempotency.
SSRF protection	Download only validated provider result hosts or use provider SDK output; cap size and timeout.
Storage	Private bucket, encryption, lifecycle rules, short-lived signed URLs. Presigned links are bearer capabilities.
Rate limiting	Per user, per IP and per payment status. Queue concurrency limits by provider.
Consent	Explicit evidence for avatars, voices, faces and uploaded assets; include takedown workflow.
Moderation	Pre-submit text moderation plus provider safety checks; store reason codes, not unnecessary sensitive text.
Audit	Record template version, provider, model, price, cost, request ID, state transitions and actor.

13. Delivery roadmap

Phase	Duration	Deliverables	Exit criteria
0. Product hardening	Week 1	Finalize template schema, refund policy, credit economics, watermark policy, consent text and provider accounts.	Eight templates versioned; unit economics signed off; test credentials available.
1. Platform foundation	Weeks 2–3	Monorepo, NestJS API, Postgres/Prisma, Firebase auth, Redis/BullMQ, local Docker, CI.	Authenticated template API and queued dummy render work end-to-end.
2. Payments + ledger	Week 4	Razorpay test checkout, verified webhook, wallet ledger, idempotent refunds.	Duplicate webhook tests pass; balances reconcile.
3. First real provider	Weeks 5–6	fal.ai/Seedance adapter, webhook/poll fallback, S3 ingestion, signed preview.	Template → payment → render → stored video succeeds reliably.
4. Frontend production wiring	Week 7	Replace timers with API status, errors, retry UX, render history.	Mobile and desktop user flows pass E2E tests.
5. Provider expansion	Weeks 8–9	Runway adapter and HeyGen adapter; provider-specific template validation.	At least one template passes quality and margin gates on each provider.
6. Admin and operations	Week 10	Template versioning UI, job console, refund action, cost/margin dashboard, alerts.	Operations can diagnose and recover failed jobs without database access.
7. Beta and launch	Weeks 11–12	Load test, security review, spend caps, backups, legal pages, monitoring, beta rollout.	SLOs, refund automation, spend alert and rollback drill verified.

14. Sprint-level task list

Sprint 1 · foundation

- Convert existing React code to TypeScript
- Create workspace packages
- Provision local Postgres and Redis
- Define Prisma schema and migrations
- Implement Firebase auth guard
- Seed eight templates

Sprint 2 · commerce

- Create Razorpay orders
- Integrate Checkout
- Verify raw-body webhooks
- Create append-only credit ledger
- Add idempotency and refund tests

Sprint 3 · rendering

- Implement provider interface
- Implement fal/Seedance adapter
- Add BullMQ worker
- Persist job events
- Download and re-host video
- Issue signed preview URLs

Sprint 4 · product

- Connect React Query

- Add render history
- Add payment and insufficient-credit UX
- Add failure/refund states
- Add admin template configuration

Sprint 5 · scale

- Runway and HeyGen adapters
- Per-provider concurrency
- Cost and margin tracking
- OpenTelemetry/Sentry
- Load, chaos and security tests

15. Testing strategy

Test level	Examples
Unit	Prompt builder, provider status mapping, price calculation, signature verification, state transition guards.
Contract	Recorded provider fixtures for submitted, processing, completed, malformed and failed responses.
Integration	Postgres transaction and ledger idempotency; Redis retry; S3 upload; signed URL.
E2E	Login → template → payment sandbox → render → preview; failure → automatic refund.
Load	Burst of render requests, provider concurrency, webhook duplication and queue recovery after restart.
Security	Broken object-level authorization, webhook forgery, SSRF, oversized scripts, secret scanning.
Financial reconciliation	Sum ledger, Razorpay settlements, provider charges and job counts for the same date window.

16. Operational SLOs and alerts

- API availability target for beta: 99.5%; public launch target: 99.9%.
- Render submission accepted within 2 seconds, excluding checkout.
- 95% of provider webhooks processed within 30 seconds of receipt.
- 100% of terminal failures create a refund decision and audit event.
- Alert when failure rate exceeds 10% in 15 minutes for a provider/model.
- Alert when actual provider cost exceeds template budget or daily spend cap.
- Dead-letter failed queue tasks and expose replay in the admin console.

17. Local development and deployment

```
# local services
services:
  postgres:
    image: postgres:17
    environment:
      POSTGRES_USER: admaya
      POSTGRES_PASSWORD: admaya
      POSTGRES_DB: admaya
    ports: ["5432:5432"]
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
```

```
# .env.example – values only belong in a secret store
DATABASE_URL=postgresql://admaya:admaya@localhost:5432/admaya
REDIS_URL=redis://localhost:6379
PUBLIC_API_URL=https://api.example.com
FIREBASE_PROJECT_ID=
FAL_KEY=
FAL_MODEL=
RUNWAYML_API_SECRET=
HEYGEN_API_KEY=
RAZORPAY_KEY_ID=
RAZORPAY_KEY_SECRET=
RAZORPAY_WEBHOOK_SECRET=
AWS_REGION=ap-south-1
S3_BUCKET=
CLOUDFRONT_DOMAIN=
```

Production deployment order

1. Create private S3 bucket, encryption and lifecycle rules.
2. Create RDS PostgreSQL with backups and private networking.
3. Create ElastiCache Redis and security groups.
4. Build API and worker container images.
5. Deploy API and worker as separate ECS services.
6. Put API behind HTTPS load balancer; configure autoscaling.
7. Configure Razorpay and provider webhook URLs.
8. Deploy React app to CloudFront/S3 or Vercel.
9. Run migrations as a controlled deployment step.
10. Turn on logs, traces, error reporting and provider spend alerts.

18. Key risks and mitigations

Risk	Impact	Mitigation
Provider model or endpoint changes	Render breakage	Adapter contract tests, pinned API versions, feature flags, fallback provider.
Temporary provider URL expires	Lost output	Download immediately and re-host in owned storage.
Duplicate callbacks	Double credits or state corruption	Unique idempotency keys and terminal-state guards.
Provider cost spike	Negative margin	Per-template cost ceiling, spend alerts, provider routing and disable switch.
Long render time	Poor UX	Async status, honest ETA ranges, notifications and cancellation policy.
Unsafe or misleading ads	Legal/reputation risk	Moderation, claims policy, audit trail and human review for sensitive categories.
Avatar consent dispute	High legal risk	Explicit consent capture, immutable evidence, takedown and retention process.
Watermark bypass	Asset leakage	Understand that client-visible media can be captured; treat watermark as deterrence, not DRM.

Critical product correction A watermark cannot guarantee that a video cannot be screen-recorded. It can discourage misuse and preserve attribution, but it is not DRM. If clean masters are sold, deliver them through authorized, expiring URLs and log each download.

19. What to do next

11. Move the current React frontend into apps/web and convert new code to TypeScript.
12. Open sandbox accounts for Razorpay, fal.ai and AWS.
13. Build the Prisma schema and seed the eight templates.
14. Implement dummy provider adapter and prove queue/state/refund logic before paying for AI renders.
15. Integrate fal.ai/Seedance for one template only: Unbox & React.
16. Upload completed video to S3 and display it via a short-lived signed URL.
17. Run ten real renders, record cost, latency and failure rate, then set the commercial price.
18. Only after the first template is reliable, add remaining templates and providers.

20. Sources and verification notes

The uploaded AdMaya.ai AI Provider Integration Playbook was used as the primary product and provider-strategy input. Current implementation details should be rechecked immediately before coding because model identifiers, pricing, quotas and payload shapes change frequently.

[1] **fal asynchronous inference**: <https://fal.ai/docs/documentation/model-apis/inference/queue>

[2] **fal webhooks**: <https://fal.ai/docs/documentation/model-apis/inference/webhooks>

[3] **Runway developer API**: <https://docs.dev.runwayml.com/>

[4] **HeyGen create video API**: <https://developers.heygen.com/reference/create-video>

[5] **Razorpay webhooks**: <https://razorpay.com/docs/webhooks/>

[6] **Amazon S3 presigned URLs**: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html>

[7] **BullMQ connections and workers**: <https://docs.bullmq.io/guide/connections>

[8] **Prisma Postgres overview**: <https://www.prisma.io/docs/postgres>

[9] Vertex AI video generation example: <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/samples/googlegenaisdk-videogen-with-img>